

Probability & Random Variables

Marginalize $P(x) = \sum_{y \in Y} P(x, y)$

$P(x|y) = \frac{P(x, y)}{P(y)}$ Chain Rule $P(x, y) = P(x|y)P(y)$

Bayes Rule

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$E[X|Y] = \sum_x x P(x|Y)$$

$X \perp Y \rightarrow P(x, y) = P(x)P(y)$
 $P(x|y) = P(x)$

X & Y are conditionally independent given z if

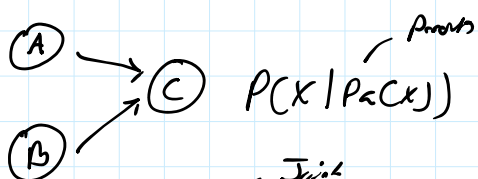
$$P(x, y|z) = P(x|z)P(y|z) \quad X \perp Y|z$$

Causal Bayesian Networks

Def of Cond. ind: $T \perp L | S$

$$P(T, L|S) = P(T|S)P(L|S)$$

$$P(T|S) = P(T|S, L)$$



$$P(x|pa(x))$$

Chain rule: $P(x_{1:n}) = \prod_{i=1}^n P(x_i | pa(x_i))$

d-Separation:

Let C be a set of R.V.'s

An undirected path between A & B is d-separated

by C if any of the follows exists:

1) Chain: $X \rightarrow Y \rightarrow Z \quad Y \in C$

d-separated: inactive

not

2) Fork: $X \leftarrow Y \rightarrow Z \quad Y \in C$

d-separated: active

3) Inverted fork: $X \rightarrow Y \leftarrow Z \quad Y \notin C$

Only need one triple to be active

3) Invert fork: $X \rightarrow Y \leftarrow Z \quad \forall \notin C$ Only need one triple to be active

We say that A & B are d -separated by C if all cyclic paths between A & B are d -separated by C

If A & B are d -separated by C , $A \perp B | C$

If there are any other paths w.r.t C between A & B , we cannot conclude $A \perp B | C$

Stochastic Processes

A stochastic process is random if

$$P(S_{t+1} | S_{0:t}) = P(S_{t+1} | S_t)$$

Decision Networks

$\square$: Decision $\circ$: Chance $\diamond$: Utility

$$E U(a|O) = \sum_{S'} P(S'|S, a) U(S'|a)$$

MDP

$$\text{Maximize } E \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad \underline{r} \leq r_t \leq \bar{r}$$

$$\frac{\underline{r}}{1-\gamma} \leq \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{\bar{r}}{1-\gamma}$$

MC Policy Evaluation $\nearrow$ trajectories

$$U(\pi) = \frac{1}{M} \sum_{i=1}^M R(\gamma^i) = \bar{v}_\pi$$

$$SEM: \frac{std(\bar{v}_\pi)}{\sqrt{M}}$$

Policy & Value Iteration

Bellman's Expectation Equation: $U^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} T(s'|s, \pi(s)) U^\pi(s')$

$$U^\pi = (I - \gamma T^\pi)^{-1} \bar{R}^\pi$$

$\hookrightarrow T(s'|s, \pi(s))$

Bellman eq: $U(s) = \max_a (R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) U(s'))$

Bellman Backup

$$U^0(s) \leftarrow 0 \quad \forall s \in S$$

Repeat:

Choose s where U^* is known for all a
Calculate $U^*(s)$

$$\pi^0(s) = \arg \max_a Q^0(s, a)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) U^*(s')$$

$$U^*(s) = \max_a Q^*(s, a)$$

Bellman's Functions:

Policy Evolution $U^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} T(s'|s, \pi(s)) U^\pi(s')$
|| optimal policy

Certifier of Optimality $U^0(s) = \max_a (R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) U^0(s'))$

Bellman's Expectation Equation

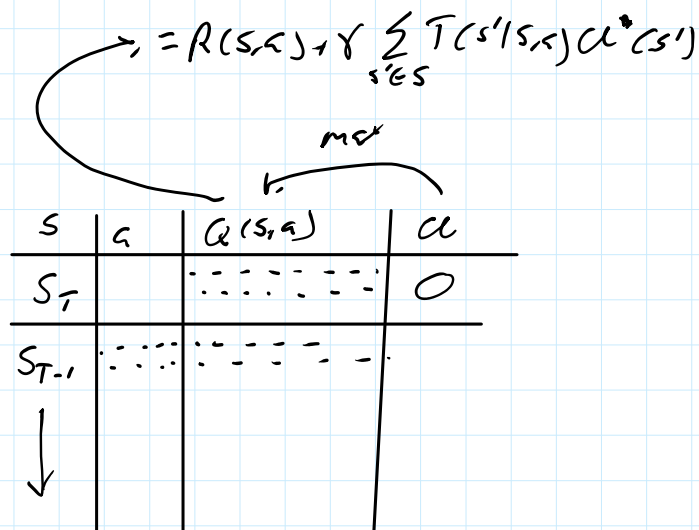
Bellman Optimality Equation

Value Iteration $U^i(s) = B[U^{i-1}](s)$
 $B[U](s) = \max_{a \in A} (R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) U(s'))$

Policy Iteration $\arg \max_{a \in A} (Q(s, a))$
 $Q(s, a) = R(s, a) + \gamma \sum_{s' \in S} T(s'|s, a) U^\pi(s')$ $\rightarrow$ matrix solve eq

Properties of MDP Solutions

- Every MDP has a unique optimal value function
- Not every MDP has a unique optimal policy
- Every MDP has a deterministic optimal policy



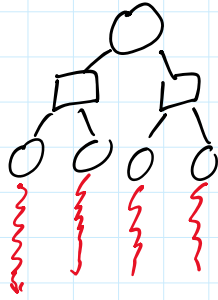
Online Methods

During Execution:

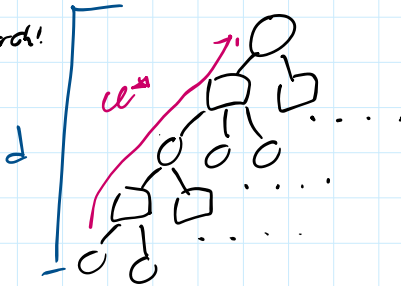
Consider actions & consequences (interval or size of 5)

Rollout Lookahead:

- Depending on rollout policy



Forward Search:



Space Savings:



MCTS/UCT

$Q(S, a)$: Value estimation
of state & action
Comb

$N(S, a)$: # of times
we visit state & action

$$Q(S, a) + \underbrace{C \sqrt{\frac{\log(N(S))}{N(S, a)}}}_{\text{Exploration Bonus}}$$

Neural Net Function Approximation

Neural Net Function Approximation

$$h(x) = \sigma(Wx + b) \quad \Theta = \{W^{(1)}, b^{(1)}, \dots, W^{(n)}\}$$

$\hookrightarrow$ N.t. fcn

Loss fcn: J N.N. $\vdots \vdots \vdots \textcircled{g} \dots 1 \dots \textcircled{g}$

$$\Theta^0 = \arg \min_{\Theta} \sum_{(x,y) \in D} J(f_{\Theta}(x), y)$$

Stochastic Gradient Descent

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} J(f_{\Theta}(x), y)$$

Continuous Space MDPs

Certainty Equivalence Principle: For Linear-Quadratic Problems, the optimal policy w/ noise is the same as the optimal policy w/out noise

Value fcn approx

$$V_{\Theta}(s) = f_{\Theta}(s)$$

boot

$$\Theta \leftarrow \Theta'$$

$$\hat{V}' \leftarrow B_{\text{opt}}[V_{\Theta}]$$

$$\Theta' \leftarrow \text{fit}(\hat{V}')$$

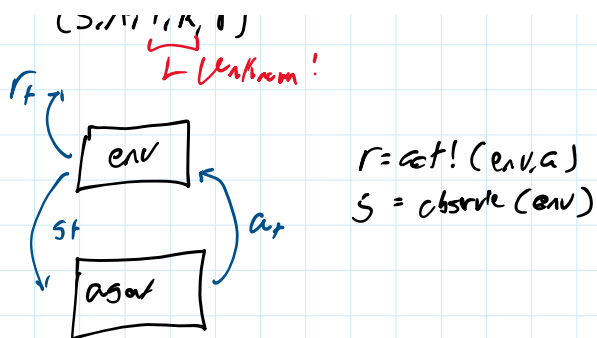
$$B_{\text{opt}}[V_{\Theta}](s) = \max_a (R(s,a) + \gamma \sum_{i=1}^n V_{\Theta}(G(s,a, \overset{\text{Weights}}{w_i}))$$

Reinforcement Learning

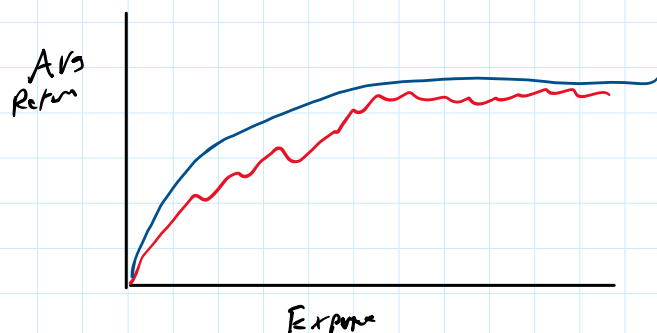
$$(S, A, \tau, R, \gamma)$$

$\tau \in \mathbb{N}$

$\hookrightarrow$ Unknown!



Learns Curve



Challenges

- 1) Exploration vs. Exploitation
- 2) Credit Assignment
- 3) Generalization

Model-Based: Learn T & R then solve MDP

Model-Free: Find Q^* or π^* directly

On-Policy: Exploration Policy is the learned policy

Off-Policy: Exploration policy may be different from learned policy

Trainer: Keep track of learned values is a task
 Deep: Use Neural Network to approximate learned values

TM2MDRL:

$$\hat{T}(s'|s, a) = \frac{N(s', a, s)}{\sum_{s'} N(s', a, s)}$$

$$\hat{R} = \frac{P(s, a)}{\sum_{s'} N(s', a, s)}$$

$\rightarrow P(s, a) = r$

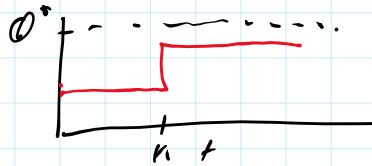
Explore & Exploitation (Bandits)

Greedy Strategies

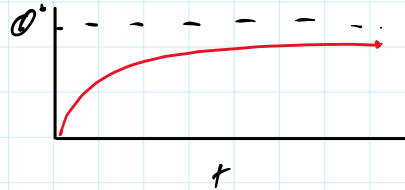
$$p_a = \frac{\# \text{ of wins} + 1}{\# \text{ of losses} + 1} \quad \text{choose } \arg \max_a p_a$$

Unstructured Strategies

- Explore then Commit
Choose a randomly for K -steps
choose $\arg \max_a p_a$

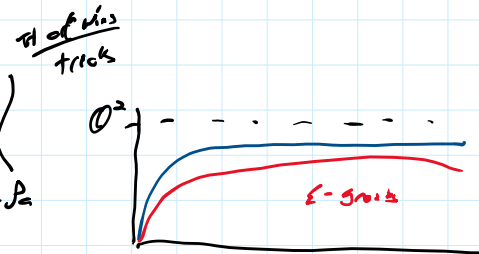


- ϵ -Greedy
w.p. ϵ , choose randomly,
otherwise choose $\arg \max_a p_a$



Directed Strategies

- Softmax
choose a w.p. $\propto e^{\lambda p_a}$



- UCB
 $\arg \max_a p_a + C \sqrt{\frac{\ln(N)}{N(a)}}$



Bayesian Estimation

Bel Distribution: Distribution over beliefs: distributions
Beta (with $J+1$)

Question Set

choose a for which a estimate of $b(\theta)$ is
highest

Thompson Sampling

h'sket
 Thompson Sample
 Sample $\hat{\theta}$
 Chern $\hat{\theta}_n$
 c

Regret Analysis

$$\text{Regret}(n) = O^{\sim} n - \sum_{t=1}^n r_t$$

		Optimal in limit	Regret
Uncached	Greedy	No	$O(n)$
	ϵ -Greedy	$\epsilon \rightarrow 0$	$O(n)$
	Follow the Count	$H \rightarrow \infty$	$O(n)$
Directed	Softmax	$\lambda \rightarrow \infty$	$O(n)$
	UCB	✓	$O(\log(n))$ ^{less}
Borison	Quantile Selection	✓	⋮
	Thompson Sampling	✓	⋮

Policy Gradient!

Policy Optimizer

$$\max_{\pi} E_{s \sim b} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_t = \pi(s_t) \right]$$

Parameterized stochastic policy

$$\max_{\theta} U(\pi_{\theta}) : U(\theta) \quad a \sim \pi_{\theta}(a|s)$$

M.C. utility

$$U(\pi) \approx \frac{1}{M} \sum_{i=1}^M R(\tau_i^{\pi})$$

trajectories

1) Zeroth order optimization

Cross entropy

2) First order optimization

Stochastic Gradient Ascent
 $\theta \leftarrow \theta + \alpha \nabla_{\theta} U(\theta)$

Tricks:

log derivative

$$\nabla_{\theta} U(\theta) = E \left[\underbrace{\sum_{n=0}^d \nabla_{\theta} \log(\pi_{\theta}(a_n | s_n)) R(\gamma)}_{\nabla_{\theta} U(\theta)} \right]$$

Policy Gradient:

loop:

$$\gamma \leftarrow \text{Sample}(\pi_{\theta})$$

$$\theta \leftarrow \theta + \alpha \sum_{n=0}^d \nabla_{\theta} \log(\pi_{\theta}(a_n | s_n)) R(\gamma)$$

On Policy

Caselli's

$$\begin{aligned} \nabla U(\theta) &= E \left[\sum_{n=0}^d \nabla_{\theta} \log(\pi_{\theta}) R(\gamma) \right] \\ &= E \left[\sum_{n=0}^d \gamma^n r_{t+g} \right] \end{aligned}$$

Basella Subtraction

$$\nabla U(\theta) = E \left[\sum_{n=0}^d \nabla_{\theta} \log(\pi_{\theta}) \gamma^n (r_{n,t+g} - r_{\text{best}}(s_n)) \right]$$

Value Based Model Free RL

Incremental Mean Estimation — Temporal Difference

Incremental Mean Estimation Temporal Difference Filter

$$\hat{x}_m = \hat{x}_{m-1} + \frac{1}{m} (x^{(m)} - \hat{x}_{m-1})$$

Some times α

Q-learning $Q(S,a) \leftarrow Q(S,a) + \alpha (r + \gamma \max_{a'} Q(S',a') - Q(S,a))$

Maximum Bias:

Even if all $Q(S',a')$ consistent, $\max_{a'} Q(S',a')$ biased!

Convergence:

Q-learning converges to optimal Q w.p. 1

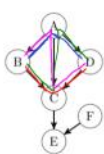
SARSA converges to optimal Q values w.p. 1

given $\pi \rightarrow$ greedy

Algorithms:

Proving Conditional Independence

1. Enumerate all (non-cyclic) paths between nodes in question
2. Check all paths for d-separation
3. If all paths d-separated, then CE



Example: $(B \perp D \mid C, E) ?$

active $\rightarrow$ inconclusive

$B \leftarrow A \rightarrow D$
 $B \rightarrow C \leftarrow D$
 $B \leftarrow A \rightarrow C \leftarrow D$
 $B \rightarrow C \leftarrow A \rightarrow D$

Separators

1. Chain: $X \rightarrow Y \rightarrow Z$ such that $Y \in C$
2. Fork: $X \leftarrow Y \rightarrow Z$ such that $Y \in C$
3. Inverted Fork (v-structure): $X \rightarrow Y \leftarrow Z$ such that $Y \notin C$ and no descendant of Y is in C .

14

Monte Carlo Policy Evaluation

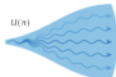
- Running a large number of simulations and averaging the accumulated reward is called *Monte Carlo Evaluation*

Let $\tau = (s_0, a_0, r_0, s_1, \dots, s_T)$ be a trajectory of the MDP

$$U(\pi) \approx \frac{1}{m} \sum_{i=1}^m R(\tau^{(i)})$$

$$\text{Var}(\bar{u}_m) = \frac{1}{m} \sum_{i=1}^m \text{Var}(\bar{u}^{(i)})$$

where $\bar{u}^{(i)}$ is generated by a rollout simulation



How can we quantify the accuracy of $\bar{u}_m$?

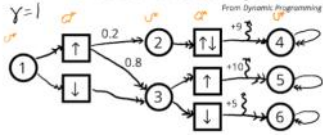
$$\begin{aligned} \text{Var}(\bar{u}_m) &= \text{Var}\left(\frac{1}{m} \sum_{i=1}^m \bar{u}^{(i)}\right) \\ &= \frac{1}{m} \text{Var}(\bar{u}^{(1)}) \\ &= \frac{1}{m} \sum_{i=1}^m \text{Var}(\bar{u}^{(i)}) \end{aligned}$$

$$\text{Var}(\bar{u}_m) = \frac{1}{m} \text{Var}(\bar{u}^{(1)})$$

$$\text{Standard Error of Mean SEM} \equiv \frac{1}{\sqrt{m}} \text{Std}(\bar{u}^{(1)})$$

15

MDP Example: Up-Down Problem



Algorithm: Bellman Backup

Given: MDP $(S, A, R, T, S_T, \gamma)$

1. $U^*(s) \leftarrow 0 \quad \forall s \in S_T$
2. Repeat until $U^*(s)$ known for all states:
 1. Choose s where U^* is known for all children
 2. Calculate $U^*(s)$
3. Extract $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$

$$U^*(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s'|s, a) U^*(s') \right)$$

$$U^*(s) = \max_a Q^*(s, a)$$

s	a	$Q^*(s, a)$	$U^*(s)$
1	up	$R(1, \text{up}) + \gamma \sum_{s'} T(s' 1, \text{up}) U^*(s') = 0 + 0.2 \cdot 10 + 0.8 \cdot 0 = 2$	0
1	down	$R(1, \text{down}) + \gamma \sum_{s'} T(s' 1, \text{down}) U^*(s') = 0 + 0.8 \cdot 10 + 0.2 \cdot 0 = 8$	0
2	up	$R(2, \text{up}) + \gamma \sum_{s'} T(s' 2, \text{up}) U^*(s') = 10 + 0.5 \cdot 0 + 0.5 \cdot 0 = 10$	9
2	down	$R(2, \text{down}) + \gamma \sum_{s'} T(s' 2, \text{down}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	9
3	up	$R(3, \text{up}) + \gamma \sum_{s'} T(s' 3, \text{up}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
3	down	$R(3, \text{down}) + \gamma \sum_{s'} T(s' 3, \text{down}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
4	up	$R(4, \text{up}) + \gamma \sum_{s'} T(s' 4, \text{up}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
4	down	$R(4, \text{down}) + \gamma \sum_{s'} T(s' 4, \text{down}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
5	up	$R(5, \text{up}) + \gamma \sum_{s'} T(s' 5, \text{up}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
5	down	$R(5, \text{down}) + \gamma \sum_{s'} T(s' 5, \text{down}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
6	up	$R(6, \text{up}) + \gamma \sum_{s'} T(s' 6, \text{up}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10
6	down	$R(6, \text{down}) + \gamma \sum_{s'} T(s' 6, \text{down}) U^*(s') = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$	10

Policy Iteration

Algorithm: Policy Iteration

Given: MDP (S, A, R, T, γ, b)

1. initialize π, π' (differently)
2. while $\pi \neq \pi'$
3. $\pi \leftarrow \pi'$
4. $U^* \leftarrow (I - \gamma T^\pi)^{-1} R^\pi$
5. $\pi'(s) \leftarrow \operatorname{argmax}_{a \in A} (R(s, a) + \gamma \sum_{s'} T(s'|s, a) U^*(s')) \quad \forall s \in S$
6. return π

(Policy iteration notebook)

Value Iteration

Algorithm: Value Iteration

Given: MDP (S, A, R, T, γ, b) , tolerance ϵ

1. initialize U, U' (differently)
2. while $\|U - U'\|_\infty > \epsilon$
3. $U \leftarrow U'$
4. $U'(s) \leftarrow \max_{a \in A} (R(s, a) + \gamma \sum_{s'} T(s'|s, a) U(s')) \quad \forall s \in S$
5. return U'

- Returned U' will be close to U^*
- π^* is easy to extract: $\pi^*(s) = \operatorname{argmax}_a (R(s, a) + \gamma E[U^*(s)])$

Contraction Mappings

Definition: A contraction mapping on metric space (M, d) is a function $f: M \rightarrow M$ satisfying

$$d(f(x), f(y)) \leq \alpha d(x, y)$$

for some $\alpha, 0 \leq \alpha < 1$ and all x and y in M .

Definition: x^* is said to be a fixed point of f if $f(x^*) = x^*$.

$$f(x) = \begin{bmatrix} \frac{x_1 + 1}{2} \\ \frac{x_2 + 1}{2} \end{bmatrix} \quad M = \mathbb{R}^2 \quad d(x, y) = \|x - y\|_2$$

$$d(f(x), f(y)) = \sqrt{\left(\frac{x_1 + 1}{2} - \frac{y_1 + 1}{2}\right)^2 + \left(\frac{x_2 + 1}{2} - \frac{y_2 + 1}{2}\right)^2} = \frac{1}{2} \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2} = \frac{1}{2} d(x, y)$$

Script: contraction_mapping.jl

f is a contraction mapping with $\alpha = \frac{1}{2}$

Banach's Theorem

Theorem (Banach): If f is a contraction mapping on metric space (M, d) , then

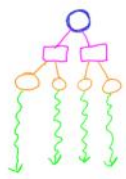
1. f has a single, unique fixed point x^* .
2. If $\{x_k\}$ is a sequence defined by $x_{k+1} = f(x_k)$, then $\lim_{k \rightarrow \infty} x_k = x^*$.

```

struct MDP
{
    # discount factor
    # state space
    # action space
    # transition function
    # reward function
    # sample transition and reward
    #  $\epsilon, \gamma, G(s, a)$ 
}

```

Rollout Lookahead



```

struct RolloutLookahead
{
    # problem
    # rollout policy
    # depth
}

function rollout(MDP, s, a) =  $\gamma V(s, a)$ 
function rollout( $\pi, s, n, d$ )
    ret = 0.0
    for t in 1:d
        s' =  $\pi(s)$ 
        r = rollout( $\pi, s', a$ )
        ret =  $\gamma V(s', t-1) + r$ 
    end
    return ret
end

function greedy(MDP, U, s)
    u, a = findmax( $\{u, \text{lookahead}(\pi, U, s, a), \pi, \theta\}$ )
    return (a, u, u)
end

function lookahead( $\pi, MDP, U, s, a$ )
     $\hat{s}, \hat{r}, \hat{u}, \hat{v} = \pi(s, a)$ 
    return  $\hat{r}(s, a) + \gamma \sum_{s'} \pi(s', a) U(s')$ 
end

```

Forward Search



Algorithm 4.6 Forward search

```

1: function SELECTACTION( $\pi, d$ )
2:   if  $d = 0$ 
3:     return (NIL, 0)
4:   ( $s', v'$ )  $\leftarrow$  (NIL, - $\infty$ )
5:   for  $a \in A(s)$ 
6:      $v \leftarrow R(s, a)$ 
7:     for  $s' \in S(s, a)$ 
8:       ( $s'', v''$ )  $\leftarrow$  SELECTACTION( $\pi', d-1$ )
9:        $v \leftarrow v + \gamma V(s'', a) v''$ 
10:    if  $v > v'$ 
11:      ( $s', v'$ )  $\leftarrow$  ( $s, v$ )
12:   return ( $s', v'$ )

```

$$(|S| \times |A|)^d$$

8.1

Sparse Sampling



Algorithm 4.8 Sparse sampling

```

1: function SELECTACTION( $\pi, d$ )
2:   if  $d = 0$ 
3:     return (NIL, 0)
4:   ( $s', v'$ )  $\leftarrow$  (NIL, - $\infty$ )
5:   for  $a \in A(s)$ 
6:      $v \leftarrow 0$ 
7:     for  $i \leftarrow 1$  to  $n$ 
8:       ( $s'', r''$ )  $\leftarrow$   $G(s, a)$ 
9:       ( $s''', v'''$ )  $\leftarrow$  SELECTACTION( $\pi', d-1$ )
10:       $v \leftarrow v + (r + \gamma v''')/n$ 
11:    if  $v > v'$ 
12:      ( $s', v'$ )  $\leftarrow$  ( $s, v$ )
13:   return ( $s', v'$ )

```

$$\frac{(n|A|)^d}{\text{size of tree}} |V^{\text{SS}}(s) - V^*(s)| \leq \epsilon \quad n, \epsilon, \text{ and } d \text{ related, but independent of } |S|$$

<https://www.cis.upenn.edu/~mkearns/papers/sparsesampling-journal.pdf>

10.1

Monte Carlo Tree Search (MCTS/UCT)

```

function (pi:MonteCarloTreeSearch)(s)
    for s in 1:n
        simulate( $\pi, s$ ) m iterations
        return argmax( $\{u, \text{lookahead}(\pi, U, s, a), \pi, \theta\}$ )
    end

function lookahead( $\pi, MDP, U, s, a$ )
    if  $d = 0$ 
        return  $\hat{r}(s, a)$ 
    end
     $\hat{s}, \hat{r}, \hat{u}, \hat{v} = \pi(s, a)$ 
    if  $\hat{u} > U(s)$ 
        for  $a' \in A(\hat{s})$ 
             $u' = \text{lookahead}(\pi, U, \hat{s}, a')$ 
             $\hat{u} = \max(\hat{u}, u')$ 
        end
    end
    return  $\hat{r}(s, a) + \gamma \sum_{s'} \pi(s', a) U(s')$ 
end

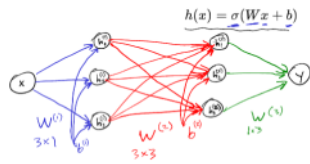
```



Visit $U(s)$
rollout gives a sample estimate of U

12.1

Neural Network



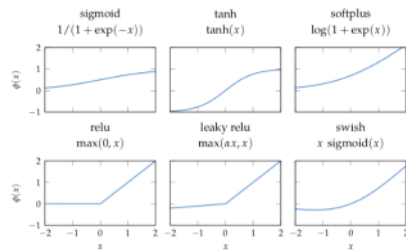
if $\sigma(x) = x$
 $y = W^*x + b^*$

$$f_{\theta}(x) = h^{(3)}(h^{(2)}(h^{(1)}(x))) = W^{(3)} \sigma^{(2)}(W^{(2)} \sigma^{(1)}(W^{(1)}x + b^{(1)}) + b^{(2)})$$

$$\theta = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, W^{(3)})$$

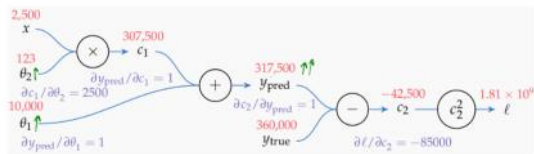
9.1

Nonlinearities



Backprop

$$l(x, y_{true}) = (\theta_2 x + \theta_1 - y_{true})^2$$



$$\nabla_{\theta} l = \begin{bmatrix} \frac{\partial l}{\partial \theta_1} \\ \frac{\partial l}{\partial \theta_2} \end{bmatrix} = \begin{bmatrix} -85,000 \\ -2,125 \times 10^8 \end{bmatrix}$$

6

1. Linear Dynamics, Quadratic Reward

$$S = \mathbb{R}^n, A = \mathbb{R}^m \quad T(s' | s, a) = \mathcal{N}(T_s s + T_a a, \Sigma) \quad R(s, a) = s^T R_s s + a^T R_a a$$

$$s' = T_s s + T_a a + w \quad w \sim \mathcal{N}(0, \Sigma) \quad (\text{Also works with other zero-mean } w.)$$

$$\text{Finite Horizon: } U_h^{\pi}(s) = \max_{\pi} \left[\sum_{t=0}^h R(s_t, a_t) \right] \quad \pi_h^* \text{ is "optimal h-step policy"}$$

$$\text{We will show that } U_h^{\pi}(s) = s^T V_h s + q_h \quad \text{and} \quad \pi_h^*(s) = -K_h s$$

22

1. Linear Dynamics, Quadratic Reward

We will show that $U_h^*(s) = s^T V_h s + q_h$ and $\pi_h^*(s) = -K_h s$ by induction.

Base: $U_1^*(s) = \max_a (s^T R_s s + a^T R_a a) = s^T R_s s$

Inductive step: show that if $U_t^* = s^T V_t s + q_t$, then $U_{t+1}^* = s^T V_{t+1} s + q_{t+1}$.

$$\begin{aligned} U_{t+1}^*(s) &= \max_a (R(s, a) + \gamma E[U_t^*(s)]) \\ &= \max_a (s^T R_s s + a^T R_a a + \int p(w) U_t(T_s s + T_a a + w) dw) \\ &= s^T R_s s + \max_a (a^T R_a a + \int p(w) (T_s s + T_a a + w)^T V_t (T_s s + T_a a + w) + q_t) dw \\ &= s^T R_s s + s^T T_s^T V_t T_s s + \max_a (a^T R_a a + 2s^T T_s^T V_t T_a a + a^T T_a^T V_t T_a a) + \int p(w) w^T V_t w dw + q_t \end{aligned}$$

$$\begin{aligned} a^* \text{ is where } \nabla_a (\text{max term}) &= 0 \\ 0 &= 2R_a a^* + 2T_a^T V_t T_s s + 2T_a^T V_t T_a a^* \end{aligned} \quad a^* = -\underbrace{(R_a + T_a^T V_t T_a)^{-1} T_a^T V_t T_s s}_{K_t}$$

$$U_{t+1}^*(s) = s^T \underbrace{(R_s + T_s^T V_t T_s - (T_a^T V_t T_s)^T (R_a + T_a^T V_t T_a)^{-1} (T_a^T V_t T_s))}_{V_{t+1}} s + \underbrace{\int p(w) w^T V_t w dw + q_t}_{q_{t+1}}$$

$$U_{t+1}^*(s) = s^T V_{t+1} s + q_{t+1} \quad \square$$

1. Linear Dynamics, Quadratic Reward

As $h \rightarrow \infty$

$$V_\infty = T_s^T (V_\infty - V_\infty T_a (T_a^T V_\infty T_a + R_a)^{-1} T_a^T V_\infty) T_s + R_s$$

$$K_\infty = (T_a^T V_\infty T_a + R_a)^{-1} T_a^T V_\infty T_s \quad \pi_\infty^*(s) = -K_\infty s$$

(K_∞ has no dependence on Σ)

Certainty-Equivalence Principle: For Linear-Quadratic problems, the optimal policy with noise is the same as the optimal policy without noise!

Practical Implication: If a continuous problem has roughly linear dynamics, a convex cost function, and roughly zero-mean additive noise, you can use *certainty-equivalent control*, i.e. control as if there is no noise.

2. Value Function Approximation

$V_\theta(s) = f_\theta(s)$ (e.g. neural network)

$V_\theta(s) = \theta^T \beta(s)$ (linear feature)

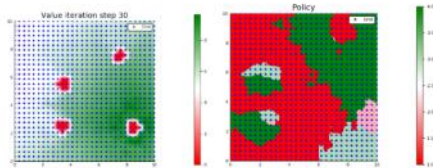
Fitted Value Iteration

while not converged

$$\theta \leftarrow \theta'$$

$$\hat{V}^t \leftarrow B_{\text{approx}}[V_\theta]$$

$$\theta' \leftarrow \text{fit}(\hat{V}^t)$$



$$B_{MC(N)}[V_\theta](s) = \max_a \left(R(s, a) + \gamma \frac{1}{N} \sum_{i=1}^N V_\theta(G(s, a, w_i)) \right)$$

3. Sparse Tree Search/Progressive Widening



add new branch if $C < kN^\alpha$ ($\alpha < 1$)



Tabular Maximum Likelihood Model-Based RL

$TMLMRRL$
 Given $(env) S, A, \gamma$
 $N[s, a, s'] \leftarrow 0 \quad \forall s, a, s'$ Number of transitions
 $R[s, a] \leftarrow 0 \quad \forall s, a$ Total reward
 $s \leftarrow observe(env)$
 $\pi \leftarrow random\ policy$
 loop
 reset!(env) $\leftarrow$ resets according to $b(s)$
 while not terminated(env)
 $a \leftarrow \begin{cases} rand(A) & w.p. \epsilon \\ \pi(a) & w.p. 1 - \epsilon \end{cases}$ $\leftarrow$ pure exploration / pure exploitation
 $r \leftarrow act(env, a)$
 $s' \leftarrow observe(env)$ stop
 $N[s, a, s'] \leftarrow N[s, a, s'] + 1$
 $R[s, a] \leftarrow R[s, a] + r$
 $s \leftarrow s'$
 $T[s, s'] \leftarrow \frac{N[s, a, s']}{\sum_{a'} N[s, a, s']}$ $\forall s, a, s'$ Maximum likelihood estimates of T and R
 $R^*[s] \leftarrow \sum_{a'} \frac{R[s, a]}{\sum_{a'} N[s, a, s']}$ $\forall s, a$
 $\pi \leftarrow solve((S, A, T, R, \gamma))$

Policy Optimization

$$\underset{\pi}{\text{maximize}} E_{s \sim b} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_t = \pi(s_t) \right] \quad \theta \rightarrow \pi_{\theta} \rightarrow U(\pi_{\theta})$$

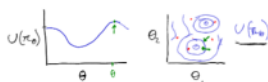
$$\underset{\pi}{\text{maximize}} U(\pi) = E_{s \sim b} [U^{\pi}(s)]$$

Two approximations:

1. Parameterized stochastic policies $\underset{\theta}{\text{maximize}} U(\pi_{\theta}) = U(\theta) \quad a \sim \pi_{\theta}(a \mid s)$
2. Monte Carlo Utility $U(\pi) \approx \frac{1}{m} \sum_{i=1}^m R(\tau^{(i)})$ trajectory: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_d, a_d, r_d)$

Two classes of optimization algorithms:

1. Zeroth order (use only $U(\theta)$)
2. First order (use $U(\theta)$ and $\nabla_{\theta} U(\theta)$)



1. Zeroth-Order Optimization

Common zeroth-order approaches:

1. Genetic Algorithms
2. Pattern Search
3. Cross-Entropy

Cross Entropy:

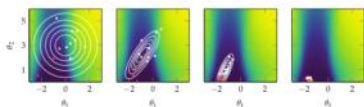
Initialize d

loop:

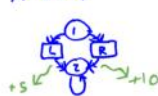
 population $\leftarrow$ sample(d)

 elite $\leftarrow m$ with highest $U(\theta)$

$d \leftarrow \text{fit}(\text{elite})$



$A = \{L, R\}$



Example

$$\pi_{\theta}(a = L \mid s = 1) = \text{clamp}(\theta, 0, 1)$$

$$\pi_{\theta}(a = R \mid s = 1) = \text{clamp}(1 - \theta, 0, 1)$$

$$a) \tau_{(1)} = (s_0=1, a_0=L, r_0=5, s_1=0, r_1=0)$$

$$V_{\theta} \log \pi_{\theta}(L \mid 1) = \frac{3}{25} \log(\text{clamp}(\theta, 0, 1)) \Big|_{\theta=0.1} = \frac{3}{25} \log(0.1) \Big|_{\theta=0.1} = \frac{1}{\theta} \log \theta \Big|_{\theta=0.1} = \frac{1}{0.1} \log 0.1 = -2.3$$

$$\widehat{V}_{\theta}(1) = \frac{1}{0.1} \cdot 5 = 5$$

$$b) \tau_{(2)} = (s_0=1, a_0=R, r_0=10, s_1=2, r_1=0)$$

$$V_{\theta} \log \pi_{\theta}(R \mid 1) = \frac{3}{25} \log(1 - \theta) \Big|_{\theta=0.1} = \frac{3}{25} \log(0.9) \Big|_{\theta=0.1} = \frac{1}{1-\theta} \log(1-\theta) \Big|_{\theta=0.1} = \frac{1}{0.9} \log 0.9 = -1.1$$

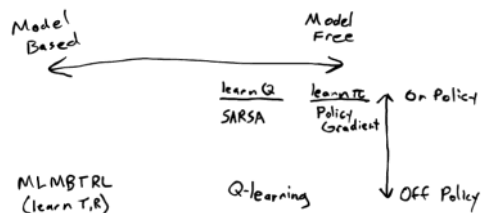
$$\widehat{V}_{\theta}(1) = \frac{1}{0.9} \cdot 10 = 11.1$$

$$\nabla U(\theta) = E \left[\sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) R(\tau) \right]$$

$$E = p_{\theta}(\tau_{(1)}) \widehat{V}_{\theta}(1) + p_{\theta}(\tau_{(2)}) \widehat{V}_{\theta}(2) = 0.1 \cdot 5 + 0.9 \cdot 11.1 = 10.0$$

Given $\theta = 0.1$ calculate $\sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) R(\tau)$ for two cases, (a) where $a_0 = L$ and (b) where $a_0 = R$

Map



Q Learning

Table $Q(s,a)$

Want $Q(s,a) \leftarrow Q(s,a) + \alpha (q(s,a,r,s') - Q(s,a))$

$Q(s,a) = R(s,a) + \gamma E[V(s)]$
 $= R(s,a) + \gamma E[\max_{a'} Q(s',a')]$

$(s,a,r,s') \leftarrow \text{experience tuple}$

$q(s,a,r,s') = r + \gamma \max_{a'} Q(s',a')$

$Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$

Q learning and SARSA

Q-Learning
 $Q(s,a) \leftarrow 0$
 $s \leftarrow s_0$
 loop
 $a \leftarrow \arg\max Q(s,a) \text{ w.p. } 1 - \epsilon, \text{ rand}(A) \text{ o.w.}$
 $r \leftarrow \text{act!}(env, a)$
 $s' \leftarrow \text{observe}(env)$
 $Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$
 $s \leftarrow s'$

SARSA
 (s,a,r,s',a') comes from exploration policy
 $a' \leftarrow \text{rand}(A)$
 $Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma Q(s',a') - Q(s,a))$
 $a \leftarrow a'$

$Q(s,a)$ $Q(s,a')$ $Q(s',a')$

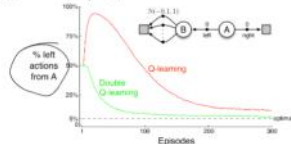
Learning Q^{optimal}

Big Problem: Maximization Bias

Even if all $Q(s',a')$ unbiased, $\max_{a'} Q(s',a')$ is biased!

Solution: Double Q Learning Q_1, Q_2

$Q_1(s,a) \leftarrow Q_1(s,a) + \alpha (r + \gamma Q_2(s', \arg\max_{a'} Q_1(s',a')) - Q_1(s,a))$
 switch at every step



SARSA-λ

$Q(s,a), N(s,a) \leftarrow 0$
 initialize s, a, r, s'
 loop
 $a' \leftarrow \arg\max Q(s',a') \text{ w.p. } 1 - \epsilon, \text{ rand}(A) \text{ o.w.}$
 $N(s,a) \leftarrow N(s,a) + 1$
 $\delta \leftarrow r + \gamma Q(s',a') - Q(s,a)$
 $Q(s,a) \leftarrow Q(s,a) + \alpha \delta N(s,a) \quad \forall s,a$
 $N(s,a) \leftarrow \gamma \lambda N(s,a)$
 $s \leftarrow s', a \leftarrow a'$
 $r \leftarrow \text{act!}(env, a)$
 $s' \leftarrow \text{observe}(env)$

N is a "decaying count of visits to (s,a) on this episode"

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Will eligibility traces work with Q-learning?

Not easily

Policy Gradient:

$$\theta \leftarrow \theta + \alpha \sum_{k=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_k | s_k) R(\tau)$$

Markov Blanket

The *Markov Blanket* of X is the minimal set of nodes that, if their values were known, would make X conditionally independent of all other nodes.

A Markov blanket of a particular node consists of its parents, its children, and the other parents of its children.

If B is the Markov blanket of X , you can treat analyze $B \cup X$ alone, and ignore any other nodes.



What is the Markov Blanket of $\{E, F\}$?

C ✓

Conditional Independence in Bayes Nets



$T \perp L | S$?
Yes

Definition of cond. indep.

$$P(T, L | S) = P(T | S) P(L | S)$$

$$P(T | S) = P(T | S, L)$$

$$P(T | S) \stackrel{?}{=} P(T | S, L)$$

$$= \frac{P(T, S, L)}{P(S, L)}$$

$$= \frac{P(T | S) P(S | L) P(L)}{\sum_i P(T=i | S) P(S | L) P(L)}$$

$$= P(T | S)$$

c) (6 pts) Suppose that you are using policy gradient with a policy parameterized with $\theta = [\theta_1, \theta_2]$ defined as

$$\pi_\theta(a = 1 | s = i) = [\theta_i]_1^1$$

where $[x]_a^b = \text{clamp}(x, a, b) = \min(b, \max(a, x))$. That is, θ_i is probability of taking action 1 in state i . If the parameter values are $\theta = [0.5, 0.5]$, what is the policy gradient estimate calculated from the trajectory above? Do not use baseline subtraction. You may find the following derivatives useful: $\frac{\partial}{\partial \theta_i} \log(\theta_i) = \frac{1}{\theta_i}$, $\frac{\partial}{\partial \theta_i} \log(1 - \theta_i) = -\frac{1}{1 - \theta_i}$.

$$c) \nabla_\theta(U) = \sum_{k=1}^3 \nabla_\theta \log \pi_\theta(a | s) \gamma^{k-1} r_{a,go}$$

step 1: $\nabla_\theta \log \pi_\theta = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \log(1 - \theta_1) \\ \frac{\partial}{\partial \theta_2} \log(1 - \theta_2) \end{bmatrix} = \begin{bmatrix} -\frac{1}{1-0.5} \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}$

$$\text{step 2: } \nabla_\theta \log \pi_\theta = \begin{bmatrix} -2 \\ 0 \end{bmatrix} \quad (\text{same as step 1})$$

$$\text{step 3: } \nabla_\theta \log \pi_\theta = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \log \theta_1 \\ \frac{\partial}{\partial \theta_2} \log \theta_2 \end{bmatrix} = \begin{bmatrix} \frac{1}{0.5} \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$r_{a,go} = 1$ (continued on next page)

Question 3b) (continued)

$$\nabla_\theta(U) = \begin{bmatrix} -2 \\ 0 \end{bmatrix} 2.71 + \begin{bmatrix} -2 \\ 0 \end{bmatrix} 0.91 + \begin{bmatrix} 2 \\ 0 \end{bmatrix} 0.1 = \begin{bmatrix} -7.04 \\ 0 \end{bmatrix}$$

$$= E \left[\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \left(\sum_{l=k}^d \gamma^l r_l \right) \right] = E \left[\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k r_{k, to-go} \right]$$

Causality

$$\nabla U(\theta) = E \left[\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) R(\tau) \right]$$

$$= E \left[\left(\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \right) \left(\sum_{k=0}^d \gamma^k r_k \right) \right]$$

$$= E \left[(f_0 + \dots + f_d) (\gamma^0 r_0 + \dots + \gamma^d r_d) \right]$$

$$= E \left[\begin{matrix} f_0 \gamma^0 r_0 + f_0 \gamma^1 r_1 + f_0 \gamma^2 r_2 + \dots + f_0 \gamma^d r_d \\ f_1 \gamma^0 r_0 + f_1 \gamma^1 r_1 + f_1 \gamma^2 r_2 + \dots + f_1 \gamma^d r_d \\ \vdots \\ f_d \gamma^0 r_0 + f_d \gamma^1 r_1 + f_d \gamma^2 r_2 + \dots + f_d \gamma^d r_d \end{matrix} \right]$$

$$= E \left[\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \left(\sum_{l=k}^d \gamma^l r_l \right) \right] = E \left[\sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k r_{k, to-go} \right]$$